

Advance Java

01 August 2026 09:12

Introduction to java & it's history

1972 -> C language (the foundation)

Goal: built an OS (like UNIX)

Limitation: C lang is platform dependent

Summary: Great for hardware but not flexible enough for networks

1990 -> The Rise of Internet (WWW)

Context: World wide web connected different types of computer globally.

The problem : We needed a language that could travel across the internet and run on any computer.

1995 -> Java (The revelation)

Solution : Released by Sun microsystem

Breakthrough : introduced WORA because of bytecode and JVM

Understanding of Java Edition (The "J2" Era)

In late 90's (specifically starting with version 1.2) java was rebranded as java 2. this is why you will often see term starting with "J2" in older documentation and legacy systems.

J2 ---> Java 2 : 20000

The Three editions:

J2SE: (Java 2 standard edition)

Morden name : JAVA SE

What it is : **"core java"** it provide the essential functionality of the language, java.lang, java.util

Usage: used for developing desktop application, command line tools

J2EE (Java 2 enterprise edition)

Morden Name: Jakarta (formally java EE)

What it is : this is "Advance Java" it built on top of J2SE to create large scale server application.

Key component: Servlet, JSP, EJP

Usage: Used for banking systems, e-commerce backend

J2ME (Java 2 micro edition)

Morden Name : Java ME

Topic: Introduction to database application & JDBC

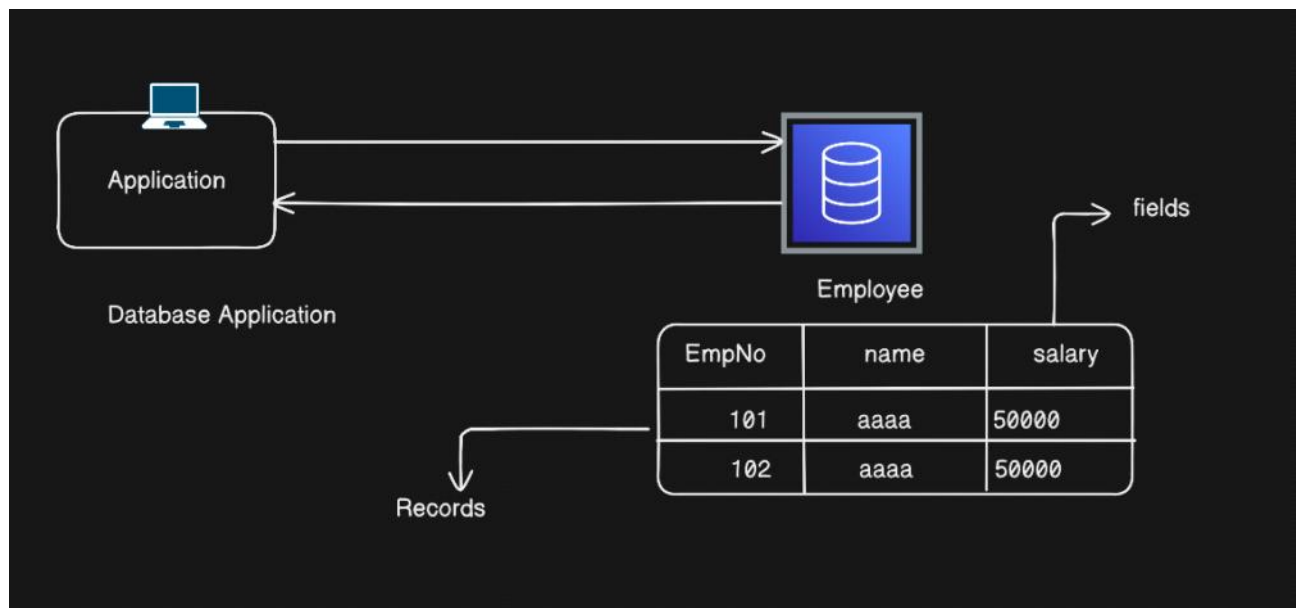
Basic Terminology:

Application: program in which we interact with on the desktop

Database: s/w which organized collection of data in some form

Database application: An application that communicates with a database

JDBC (Java Database connectivity)



Database software contains two part:

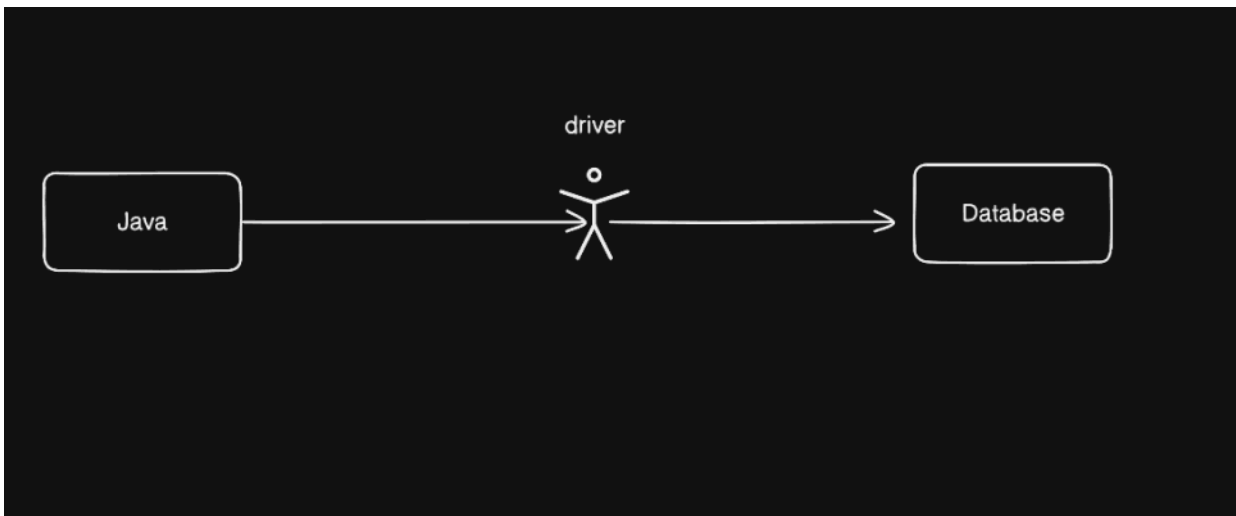
1. Database application
2. Database

Note: Database app is front end where database is backend application and both are communicating.

SQL: SQL stand for structure query language and it used by application to communicate database.

Driver:

A driver is s/w and it used to connect application and database.

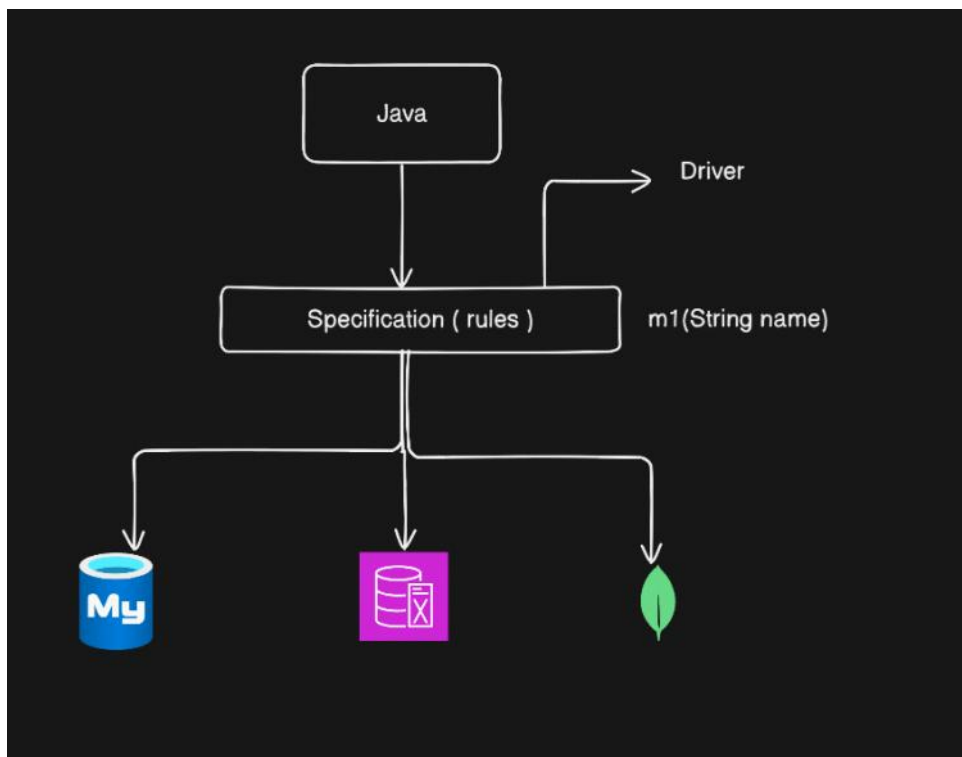


Driver are developed by first party vendor, second party vendor and third party vendor.

First Party Vendor:

The company that designed the programming language and specification (rule) for connection

Ex: sun microsystem



Second party vendor (creator of database)

The company that creates the database software

Ex: oracle corporation

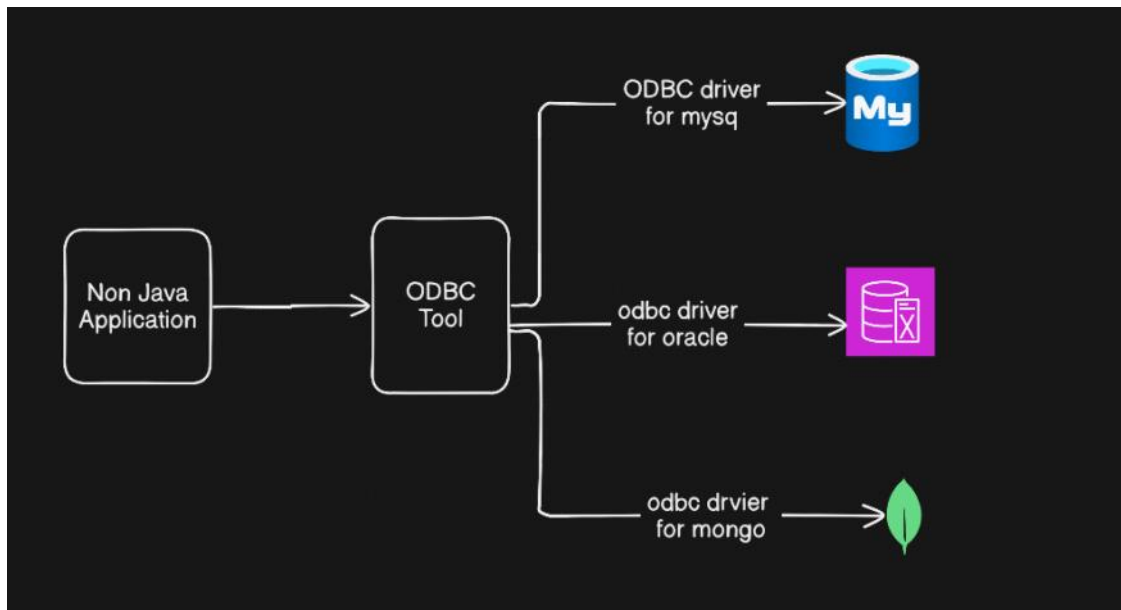
Third party vendor: (external facilitator)

Any company other than first party or second party that create tools or lib to help in making connection

There are two type of driver:

1. ODBC (open database connectivity) drivers
2. JDBC (java database connectivity) drivers

ODBC is used to connect non java application & database, ODBC driver exists before java.



Why JDBC ?

1. Java cannot directly use ODBC
2. ODBC does not understand java instruction
3. JDBC was introduced to connect java application with databases.

JDBC Drivers:

There are 4 types of JDBC drivers:

1. Type-1: JDBC-ODBC bridge (Deprecated)
2. Type-2 : Native API driver
3. Type-3: Network protocol Driver
4. Type-4: Pure Java (thin driver, Most used)

Type 1-JDBC-ODBC Bride

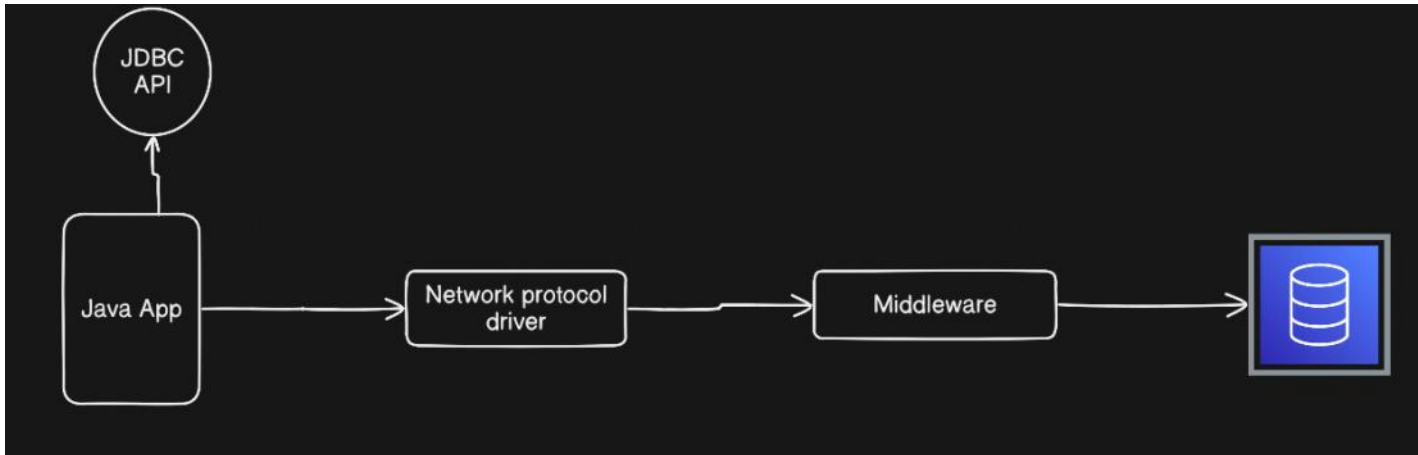
Driver class name : sun.jdbc.odbc.jdbcodbcDriver.
Removed in 1.8

Type-2 Native Api Driver

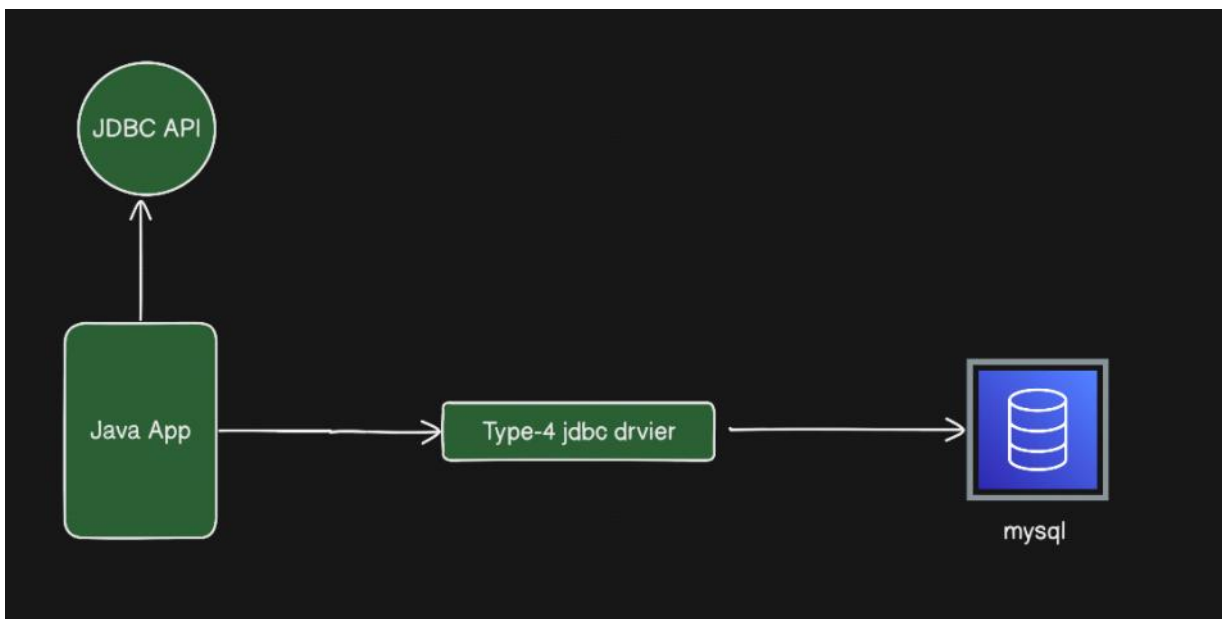
It is also called partly java, partly native driver or partial java driver.

Type-3 JDBC Driver

Type-3 JDBC driver sends JDBC calls to a middleware server and that server talk to database



Type-4 JDBC driver (Pure java driver)



The java.sql.Connection Interface

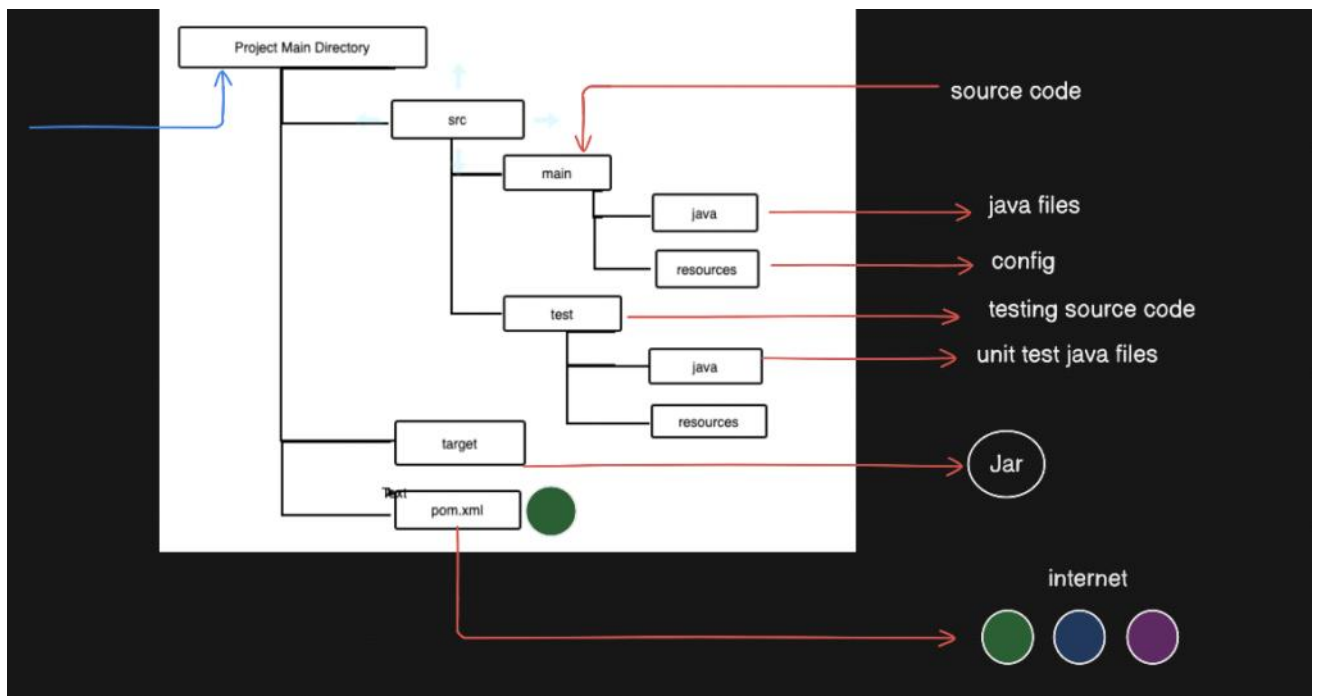
Think of the connection as a Phone line between you java program and the database. Once the line is open, you need to decide how you want to speak. You have three options (methods):

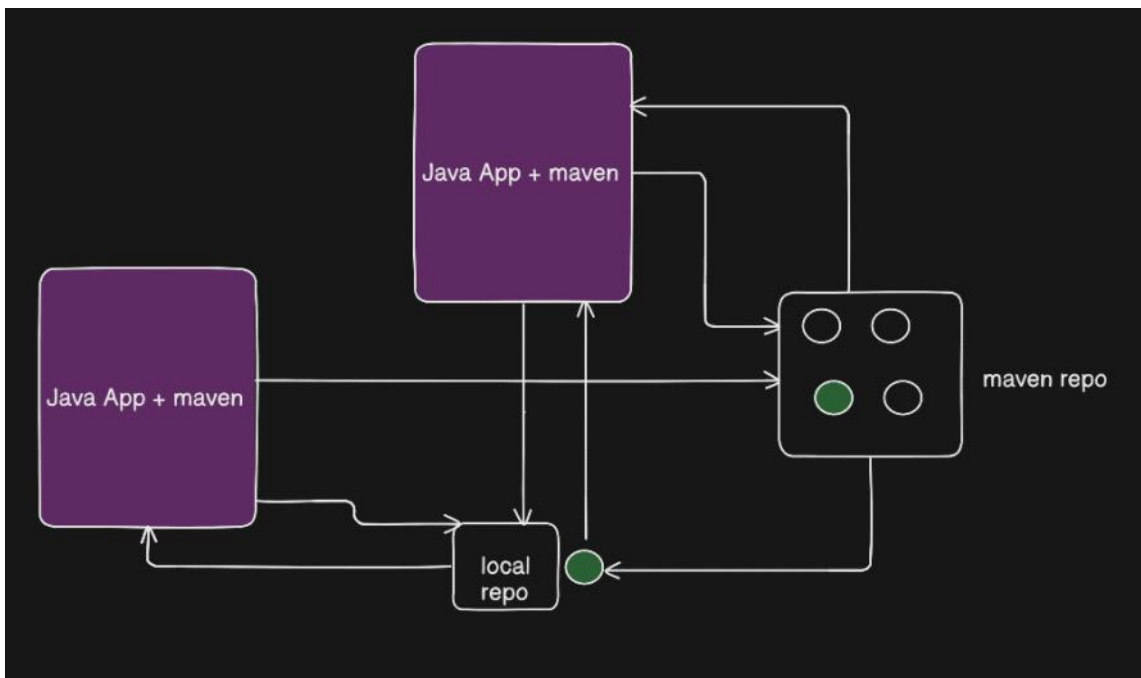
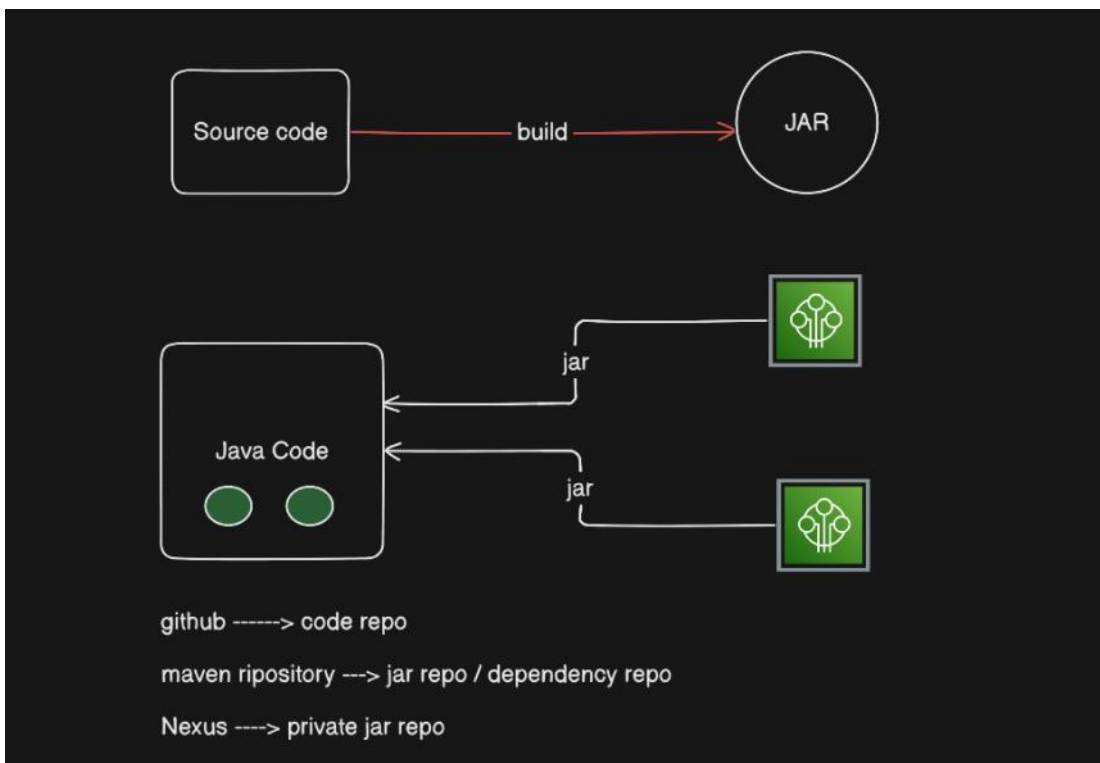
1. Statement interface
 - a. Defination: it is used to execute sql queries
 - b. Example: Select * from employees
2. PreparedStatement interface
 - a. Defination: it is used to execute dynamic sql queries.
3. CallableStatement interface.
 - a. Defination: it is used to execute PL/SQL programs.

1. Execute(): use for DDL queries
 - a. Eg: CREATE, ALTER, DROP
2. executeUpdate() : DML queries
 - a. Eg: INSERT, UPDATE, DELETE
3. executeQuery() : DQL queries
 - a. EG: select

P01: connection establish

Maven:





Statement interface:

It is used to execute static sql queries.

Ex: select * from emp
createStatement();

PreparedStatement Interface

It is used for execute dynamic queries.

Ex: select * from employee where = ?
prepareStatement(sql)

1. Execute() : DDL---> create, alter , drop
2. executeUpdate() : DML ---> insert, update, delete
3. executeQuery(): DQL ---> select

ResultSet interface

When you run a SELECT query the database sends back a table of data in java this table is stored in an object called resultset.

LLD:

1. Add a student
2. View all student
3. Search student by id
4. Update student details
5. Delete student
6. Exit the application